

INTERNATIONAL INTERNSHIP WORK REPORT

Name

Nonpravit Kachonnarongvanich

School of Engineering,

King Mongkut's Institute of Technology Ladkrabang

Name of Institution or company	The University of Electro-Communications
City and Country	Chofu, Tokyo, Japan
Period of Internship	18 May 2026 – 30 June 2026
Research Topic or Activity Name	Implementation of a Post-Quantum Key-Exchange Algorithm
Host Advisor's Name	Prof. Bagus Santoso

Acknowledgements

This research was carried out at the University of Electro-Communications (UEC) in Japan under the supervision of Prof. Bagus Santoso. I am deeply grateful to Prof. Santoso for taking the time to explain the mathematical background of post-quantum cryptography, especially the Learning Parity with Noise problem, and for providing clear direction throughout the project. His feedback at every stage was essential to achieving the results in this report.

I also thank the members of the laboratory at UEC who were always ready to help and answer questions. Working in this environment gave me a much better understanding of what research looks like in practice.

This project pushed me to learn topics I had not studied before — lattice-based cryptography, GPU computing with PyTorch and CUDA, and large-scale numerical experimentation. The knowledge I gained here will stay with me long after this internship ends.

Last, I thank my family and friends for their patience and support during the weeks I spent running experiments and writing this report.

Nonpravit Kachonnarongvanich

June 2026

Abstract

This report presents the implementation and evaluation of a post-quantum key exchange algorithm based on the Learning Parity with Noise (LPN) problem. The protocol allows two parties to independently derive the same shared secret key over a public channel. Because the security relies on the hardness of the LPN problem — for which no efficient quantum algorithm is known — the approach remains secure even against quantum computer attacks.

We implemented the full key exchange protocol and used a majority voting technique to overcome the key agreement error caused by the LPN noise. Two experimental tasks were conducted. Task 1 applies a single majority function over a 1024-bit key vector and achieves approximately 53.8% per-run success. Task 2 applies majority of majority across 10,000 runs per experiment and achieves approximately 98% agreement between both parties.

The entire computation was implemented in Python using PyTorch and accelerated on a CUDA GPU. Running 10,000 experiments of 10,000 runs each required approximately 19 hours. Both experimental results match theoretical predictions and confirm that LPN-based key exchange is mathematically feasible with the chosen parameters ($m=512$, $n=256$, $k=16$, $l=1024$).

Table of Contents

Chapter 1	Introduction	5
Chapter 2	The LPN Problem.....	6
Chapter 3	Key Exchange Protocol.....	7
Chapter 4	Implementation with PyTorch and CUDA.....	9
Chapter 5	Experiments and Results	12
Chapter 6	Conclusion and Future Work	15

Chapter 1 Introduction

1.1 Background: Why Key Exchange?

In symmetric-key cryptography, two communicating parties must share the same secret key before they can encrypt and decrypt messages. The fundamental challenge is: how can two parties agree on a shared key over an insecure public channel without an attacker intercepting it? This is known as the Key Distribution Problem, and Key Exchange (KE) algorithms solve it.

Modern key exchange algorithms are widely deployed in everyday devices — HTTPS connections, Wi-Fi security, and mobile communications all rely on them. However, virtually all current algorithms are based on mathematical problems such as integer factorization or discrete logarithms, which a sufficiently powerful quantum computer could solve efficiently using Shor's algorithm.

1.2 The Quantum Threat

If a large-scale quantum computer becomes available, most currently deployed key exchange systems would be broken. This motivates the field of Post-Quantum Cryptography (PQC) — the design of cryptographic algorithms that remain secure even against quantum adversaries.

1.3 Our Approach: LPN-Based Key Exchange

In this project, we implement a key exchange algorithm based on the Learning Parity with Noise (LPN) problem. LPN is believed to be computationally hard even for quantum computers, making it a strong candidate for post-quantum security. The implementation was accelerated using PyTorch on a CUDA GPU to handle the large number of experiments required for evaluation.

Chapter 2 The LPN Problem

2.1 Definition

The Learning Parity with Noise (LPN) problem is defined as follows. Given publicly known values:

- A — a full-rank binary matrix of size $m \times n$ (all elements are 0 or 1)
- U — a binary matrix of size $m \times 1$
- k — a natural number less than m (the sparsity parameter)

The task is to find binary matrices S (size $n \times 1$) and E (size $m \times 1$) such that:

$$U = A \cdot S \oplus E \pmod{2}$$

where each column of the error matrix E has exactly k non-zero elements. The matrix E is called the sparse error matrix. The sparsity constraint means E is mostly zeros with only k ones per column.

2.2 Hardness of LPN

Without knowing S , finding E from the publicly available A and U is computationally infeasible when m and n are sufficiently large. This hardness holds for classical computers and, crucially, no efficient quantum algorithm is known for solving LPN. This makes LPN a promising foundation for post-quantum cryptographic protocols.

The difficulty arises because the search space for S is exponentially large, and the sparse structure of E provides insufficient information to efficiently recover S without prior knowledge. The problem has been studied for several decades and its hardness is well-supported by the cryptographic research community.

2.3 Parameters Used

In this project, the following parameters were used:

Parameter	Symbol	Value
Matrix rows	m	512
Matrix columns	n	256
Sparsity (ones per error column)	k	16
Key vector length	l	1,024

Table 2.1 Experimental parameters

Chapter 3 Key Exchange Protocol

3.1 Protocol Overview

The LPN-based key exchange protocol allows two parties, Party 1 (P1) and Party 2 (P2), to independently derive the same secret key over a public channel. Both parties start with a shared public matrix A . The protocol proceeds as follows:

Party 1 (P1)	Party 2 (P2)
Generate private: $S1$, $E1$ ($k=16$ ones/col), $f1$	Generate private: $S2$, $E2$ ($k=16$ ones/col), $f2$
$b1 = A^T f1 \pmod{2}$ $U1 = A \cdot S1 \oplus E1 \pmod{2}$	$b2 = A^T f2 \pmod{2}$ $U2 = A \cdot S2 \oplus E2 \pmod{2}$
$\longleftrightarrow$ Exchange ($U1$, $b1$) and ($U2$, $b2$) over public channel $\longleftrightarrow$	
$K1 = U2^T f1 \oplus S1^T b2 \pmod{2}$	$K2 = U1^T f2 \oplus S2^T b1 \pmod{2}$

Table 3.1 LPN-based key exchange protocol steps

3.2 Key Agreement Error

After the protocol, Party 1 holds key vector $K1$ and Party 2 holds $K2$, each of length $l = 1024$ bits. Due to the sparse error matrices $E1$ and $E2$, $K1$ and $K2$ differ in some bits. The difference can be expressed as:

$$K1 \oplus K2 = E2^T f1 \oplus E1^T f2 \pmod{2}$$

When tested without any correction, the probability that $K1$ exactly equals $K2$ is essentially 0%. In 10,000 experiments, not a single exact match was observed. A correction mechanism is therefore required.

3.3 Majority Function

The majority function is applied to reconcile the differences between $K1$ and $K2$. Given a binary vector $x = (x_1, x_2, \dots, x_l)$ where each $x_i \in \{0, 1\}$:

$$\begin{aligned} \text{maj}(x) &= 1 \quad \text{if } \#\{i : x_i = 1\} \geq l/2 \\ \text{maj}(x) &= 0 \quad \text{otherwise} \end{aligned}$$

Instead of comparing $K1$ and $K2$ element by element, each party applies the majority function to their entire key vector to produce a single bit. Since more than 50% of the bits

of K_1 and K_2 agree (even with the error), the majority vote produces the same single bit for both parties most of the time, amplifying the small agreement advantage into a reliable shared key bit.

Chapter 4 Implementation with PyTorch and CUDA

4.1 Development Environment

The implementation was developed in Python 3.12 using PyTorch as the main numerical computation library. All matrix operations were performed on a CUDA-compatible NVIDIA GPU to achieve the computational performance required for 10,000 experiments $\times$ 10,000 runs each.

Component	Details
Language	Python 3.12
Deep Learning Framework	PyTorch 2.x
GPU Acceleration	CUDA (NVIDIA GPU)
Output Format	CSV files + Excel (.xlsx)
Total Runtime	~19 hours on GPU

Table 4.1 Development environment

4.2 GPU Initialization and Matrix Generation

The implementation begins by setting the computation device to CUDA if a compatible GPU is available. All tensors are allocated directly on the GPU to avoid expensive CPU-GPU data transfers during computation.

```
import torch
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

The public matrix A is generated once and reused across all runs within an experiment. It is a random binary matrix of size $m \times n$:

```
A = torch.randint(0, 2, (m, n), dtype=torch.float32, device=device)
```

For each run, the secret matrices $S1$, $S2$ (size $n \times 1$) and sparse error matrices $E1$, $E2$ (size $m \times 1$) are generated. Each column of E contains exactly $k = 16$ randomly placed ones:

```
# Generate sparse error matrix with exactly k ones per column
```

```
def sparse_error(m, l, k, device):
    E = torch.zeros(m, l, device=device)
    for j in range(l):
        idx = torch.randperm(m, device=device)[:k]
        E[idx, j] = 1
    return E
```

4.3 Key Exchange Computation

The key exchange formulas are implemented using PyTorch matrix multiplication with mod-2 arithmetic. The @ operator is used for matrix multiplication, and % 2 converts results to binary:

```
# Party 1 and Party 2 compute b and U
b1 = (A.T @ f1) % 2      # AT f1 mod 2
U1 = (A @ S1 + E1) % 2  # AS1 ⊕ E1 mod 2
b2 = (A.T @ f2) % 2
U2 = (A @ S2 + E2) % 2

# Both parties compute their key vectors
K1 = (U2.T @ f1 + S1.T @ b2) % 2  # 1024-bit vector
K2 = (U1.T @ f2 + S2.T @ b1) % 2  # 1024-bit vector
```

The majority function is applied to reduce each 1024-bit key vector to a single bit:

```
maj_K1 = (K1.sum() >= 1 / 2).int().item()  # 0 or 1
maj_K2 = (K2.sum() >= 1 / 2).int().item()  # 0 or 1
```

4.4 Batched Processing

To maximize GPU utilization, runs are processed in batches of BATCH = 50. Multiple sets of S, E, and f matrices are stacked into 3D tensors and processed simultaneously, significantly reducing runtime compared to processing one run at a time:

```
BATCH = 50
# Stack BATCH copies: shape (BATCH, n, l)
S1_batch = torch.randint(0, 2, (BATCH, n, l), device=device)
E1_batch = sparse_error_batch(m, l, k, BATCH, device)
f1_batch = torch.randint(0, 2, (BATCH, m, l), device=device)

# Batched matrix operations using torch.bmm()
b1_batch = torch.bmm(A.T.unsqueeze(0).expand(BATCH, -1, -1), f1_batch) % 2
U1_batch = (A.unsqueeze(0) @ S1_batch + E1_batch) % 2
```

4.5 Algorithm Flowchart

Figure 4.1 shows the complete algorithm flowchart, illustrating the overall structure from GPU initialization through the nested experiment and run loops to the final result computation.

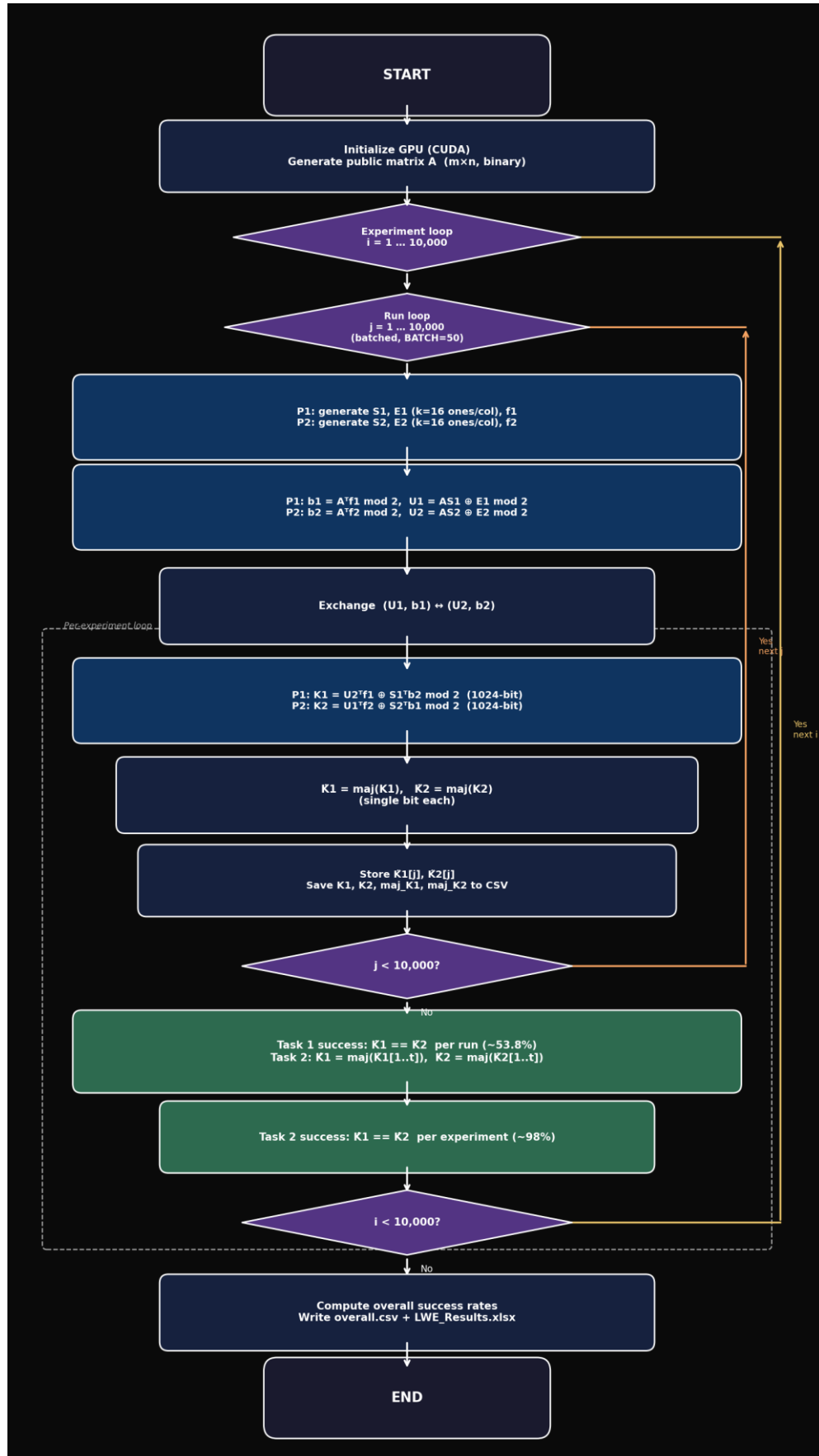


Figure 4.1 Algorithm flowchart for the LPN-based key exchange implementation

Chapter 5 Experiments and Results

5.1 Experimental Setup

Two experimental tasks were conducted to evaluate the effectiveness of the majority voting approach. Both tasks use the same parameters ($m=512$, $n=256$, $k=16$, $l=1024$) and run 10,000 experiments with 10,000 runs per experiment. The difference lies in how the majority function is applied.

5.2 Raw Protocol — Baseline

Before applying any majority function, the raw probability of exact key agreement ($K1 = K2$ element-by-element) was tested. The result was approximately 0% — no exact matches were observed in 10,000 experiments. This confirms that the error matrices $E1$ and $E2$ cause sufficient bit differences to make direct comparison impossible.

5.3 Task 1 — Single Majority

In Task 1, the majority function is applied once to the full 1024-bit key vectors to produce a single bit per run:

$$\tilde{K}1 = \text{maj}(K1), \quad \tilde{K}2 = \text{maj}(K2)$$

Success is defined as $\tilde{K}1 = \tilde{K}2$ for a given run. The experiment was repeated 10,000 times.

Result: The per-run success rate was approximately 53.8%. This is meaningfully above 50% (random guessing), confirming that with $k=16$ and $l=1024$, the majority function provides a genuine advantage. However, 53.8% is not reliable enough for practical key exchange.

5.4 Task 2 — Majority of Majority

In Task 2, the protocol is run 10,000 times per experiment. Each run produces $\tilde{K}1$ and $\tilde{K}2$ as single bits. These 10,000 bits are then aggregated and a second majority vote is applied:

$$\begin{aligned} \hat{K}1 &= \text{maj}(\tilde{K}1(1), \tilde{K}1(2), \dots, \tilde{K}1(10000)) \\ \hat{K}2 &= \text{maj}(\tilde{K}2(1), \tilde{K}2(2), \dots, \tilde{K}2(10000)) \end{aligned}$$

Success is defined as $\hat{K}1 = \hat{K}2$ per experiment. The result was approximately 98%.

This dramatic improvement from 53.8% to 98% is explained by the Law of Large Numbers: when each trial is independently correct with probability $53.8\% > 50\%$, the majority of 10,000 such trials is correct with very high probability. The nested majority effectively amplifies a small per-bit advantage into a highly reliable shared key bit.

5.5 Summary of Results

Method	Parameters	Success Rate	Practical?
Raw Protocol	$k=16, l=1024$	$\sim 0\%$	No
Task 1: Single Majority	$k=16, l=1024$	$\sim 53.8\%$	No
Task 2: Majority of Majority	$k=16, l=1024$	$\sim 98\%$	Yes

Table 5.1 Experimental results summary

Chapter 6 Conclusion and Future Work

6.1 Conclusion

This project successfully implemented a post-quantum key exchange algorithm based on the Learning Parity with Noise (LPN) problem. The implementation was accelerated using PyTorch and CUDA on a GPU, enabling 10,000 experiments of 10,000 runs each.

The results demonstrate a clear progression through three methods:

1. Raw protocol: $\sim 0\%$ exact key agreement due to the LPN error term.
2. Task 1 (single majority): $\sim 53.8\%$ per-run agreement, confirming a genuine advantage over random guessing.
3. Task 2 (majority of majority): $\sim 98\%$ agreement, confirming that nested majority effectively amplifies the small per-run advantage into a reliable shared key bit.

These results match theoretical predictions and confirm that LPN-based key exchange is mathematically feasible with the parameters $m=512$, $n=256$, $k=16$, and $l=1024$. The approach successfully demonstrates that post-quantum key exchange can be implemented in practice, although significant engineering challenges remain before it can be deployed in real-world systems.

6.2 Future Work

The following areas are identified for future research and improvement:

4. Speed optimization: The current implementation required approximately 19 hours of GPU computation to produce one reliable key bit. For practical deployment, the algorithm must be significantly faster. Future work should explore more efficient algorithms, optimized CUDA kernels, and hardware-specific acceleration.
5. Larger parameters: The parameters $m=512$ and $n=256$ are too small for real-world security. Real-world LPN-based systems require much larger matrices, which will further increase computational requirements. Research into parameter selection that balances security and performance is needed.

6. Multi-bit key generation: This project produces only a single key bit per protocol execution. Extending the protocol to generate a full multi-bit key efficiently is necessary for practical cryptographic use.
7. Hardware acceleration: Future implementations should explore FPGA-based or ASIC-based hardware acceleration to drastically reduce computation time and enable real-time key exchange.
8. Security analysis: A formal security analysis of the majority-of-majority approach against known LPN attacks should be conducted to quantify the exact security level provided by the chosen parameters.

Reference

The Organic Chemistry Tutor. (2018, February 17). *Multiplying matrices* [Video]. YouTube. <https://www.youtube.com/watch?v=vzt9c7iWPxs>

Juddy Productions. (2018, July 22). *permutation matrices* [Video]. YouTube. <https://www.youtube.com/watch?v=LVY1iUZZ-q0>

blackpenredpen. (2020, October 8). *Properties of the transpose of a matrix, linear algebra tutorial* [Video]. YouTube. <https://www.youtube.com/watch?v=HgFUYepT7FM>

Host's advisor signature

()

Date.....

